



ECHO"NAME : ADHM AHMED ALI"



ECHO"TRACK : DATA ENGINEERING"



ECHO"LOCATION : EL FAYOUM"



DATAWAREHOUSE



Energy Utility Customer BI Project

Comprehensive Technical Guide

OLTP → ODS → Data Warehouse → SSAS Cube → Dashboard

Layer	Technology	Database / Tool
OLTP Source	SQL Server / SSMS	EnergyUtility_TransactionDB
ODS / Staging	SQL Server + SSIS	EnergyUtility_StagingDB
Data Warehouse	SQL Server star schema	EnergyUtility_DataWarehouse
Data Mart	SQL Server	EnergyUtility_DataMart_Billing
ETL Pipeline	SSIS (4 packages)	Visual Studio SSDT
OLAP Layer	SSAS Multidimensional	EnergyUtility_SSAS
Dashboard	Excel + Power BI	Live connection to SSAS
Scheduling	SQL Server Agent	Nightly 2 AM automated run

Total source rows: ~393,000 • Star schema: 6 dims + 2 facts • MDX calculated members: 15+

Table of Contents

Table of Contents.....	3
1. Project Overview	6
1.1 Business Questions Answered	6
1.2 Full Architecture Flow	6
1.3 File Inventory	6
2. OLTP Source — EnergyUtility_TransactionDB	8
2.1 Table Groups and Row Counts	8
2.2 Core Entity Tables	8
Customer	8
Tariff.....	8
ServiceAccount	9
Meter.....	9
2.3 High-Volume Tables	9
MeterReading	9
CustomerMeter	9
2.4 Finance Tables	9
Billing	9
Payment.....	9
BillingDispute	10
2.5 Incremental Load Strategy per Table	10
3. ODS / Staging — EnergyUtility_StagingDB	11
3.1 Design Principles	11
3.2 Audit Columns on Every Raw Table.....	11
3.3 ETL Control Tables	12
ETL_Watermark.....	12
ETL_RunLog.....	12
4. Data Warehouse — EnergyUtility_DataWarehouse.....	13
4.1 SCD Strategy.....	13
4.2 SCD Type 2 Columns (on Customer, ServiceAccount, Meter)	13
4.3 DimDate — Pre-populated Calendar.....	14
4.4 FactEnergyUsage	14
4.5 FactBilling	15
5. Billing Data Mart — EnergyUtility_DataMart_Billing	16
5.1 Key Differences from the DW.....	16
5.2 Tables.....	16

6. SSIS ETL Pipeline — Four Packages.....	17
6.1 Package 1 — OLTP to Staging	17
What it does	17
Control flow — task sequence	17
Parameter mapping for every staging task (Tasks 2-13)	18
Error handling	18
6.2 Package 2 — Staging to Dimensions (SCD-2)	18
SCD-2 MERGE logic (two-step)	18
6.3 Package 3 — Staging to Facts.....	19
FactEnergyUsage	19
FactBilling	19
6.4 Package 4 — DW to Data Mart.....	19
6.5 Verification Queries After Each Package	19
After Package 1.....	19
After Package 2.....	19
After Package 3.....	20
After Package 4.....	20
7. SSAS Multidimensional Cube — EnergyUtility_SSAS	21
7.1 Build Order in Visual Studio	21
7.2 Six Dimensions	21
7.3 Dimension Usage — EnergyUsageCube	22
7.4 Dimension Usage — BillingCube	22
7.5 MDX Calculated Members — EnergyUsageCube	23
7.6 MDX Calculated Members — BillingCube	23
7.7 KPIs (BillingCube).....	24
7.8 Nightly Cube Refresh.....	24
8. Excel Dashboard	25
8.1 Workbook Structure	25
8.2 Slicer Connection Rule	25
8.3 Conditional Formatting for KPI Columns	25
8.4 Auto-Refresh Setup	26
9. Power BI Dashboard	26
9.1 Report Pages per File	26
EnergyUsage_Report.pbix	26
Billing_Report.pbix	26
9.2 Automated Full Refresh Chain	27

10. Complete Run Order	28
10.1 One-time Setup (SSMS — run scripts in this exact order)	28
10.2 SSIS (Visual Studio — run packages in this order after building them)	28
10.3 SSAS (Visual Studio)	28
10.4 SQL Server Agent Job	29
10.5 Troubleshooting Common Errors	29
Appendix A — Skills Practised	30
SQL Server Management Studio (SSMS)	30
SSIS (Visual Studio SSDT)	30
Data Modelling	30
SSAS Multidimensional	30
Excel and Power BI	31
Appendix B — Seed Data Distributions	31

Section 1

Project Overview and Architecture

1. Project Overview

This project builds a complete on-premises Business Intelligence stack for an energy utility company using the Microsoft SQL Server toolchain. The goal is to take raw transactional data from an OLTP source database and transform it — through a staging layer, a star-schema data warehouse, and an SSAS multidimensional cube — into actionable analytical dashboards for three business teams: operations, finance, and executive leadership.

1.1 Business Questions Answered

- How much energy does each customer use, and when during the day?
- What is our peak-load exposure by customer type and geography?
- What percentage of invoices are collected on time?
- Which customers are overdue, in dispute, or at collection risk?
- How do billing amounts and collection rates trend month over month?
- Are there meter anomalies that indicate faulty readings or fraud?

1.2 Full Architecture Flow

OLTP (TransactionDB) → SSIS Pkg1 → Staging (StagingDB) → SSIS Pkg2/3 → Data Warehouse → SSIS Pkg4 → Data Mart → SSAS ProcessFull → SSAS Cube → Excel / Power BI

1.3 File Inventory

File	Target Database	Purpose
01_OLTP_Schema.sql	TransactionDB	Creates all 12 OLTP tables with constraints and triggers
02_OLTP_SeedData_v2.sql	TransactionDB	Inserts ~393K realistic rows across all tables

File	Target Database	Purpose
03_Staging_Schema.sql	StagingDB	Creates 12 Raw tables + ETL_RunLog + ETL_Watermark
04_DW_Schema.sql	DataWarehouse	Creates 6 dimension tables + 2 fact tables + DimDate populated
05_DataMart_Schema.sql	DataMart	Creates 3 finance-optimised mart tables
06_Package1_Staging_SPs.sql	StagingDB	Stored procs for incremental OLTP extraction
07_Package2_Dims_SPs.sql	DataWarehouse	Stored procs for SCD-2 dimension loading
08_Package3_Facts_SPs.sql	DataWarehouse	Stored procs for FactEnergyUsage and FactBilling loading
09_Package4_DataMart_SPs.sql	DataMart	Stored procs for billing mart refresh
10_SSIS_Wiring_Guide.txt	Visual Studio	Step-by-step SSIS package build instructions
11_SSAS_Setup_Guide.txt	Visual Studio	SSAS project, dimensions, cubes, deploy guide
12_SSAS_MDX_Calculations.sql	SSAS	All calculated members, KPIs, and named sets
13_Excel_Dashboard_Guide.txt	Excel	PivotTable layout and dashboard build guide
14_PowerBI_Dashboard_Guide.txt	Power BI	Power BI visuals, gateway, and publish guide

Section 2

Phase 1 — OLTP Source Database

2. OLTP Source — EnergyUtility_TransactionDB

The OLTP database is the transactional system of record. It contains 12 tables organised into four functional groups. All tables follow the same audit pattern: every table has CreatedAt and most have ModifiedAt to support incremental ETL extraction.

2.1 Table Groups and Row Counts

Group	Tables	Approx. Rows
Core entities	Customer, Tariff, ServiceAccount, Meter	3,005
High-volume readings	MeterReading, CustomerMeter	~366,000
Finance and billing	Billing, Payment, BillingDispute	~20,860
Operational records	MeterInstallation, MeterMaintenance, Notification	4,200

2.2 Core Entity Tables

Customer

Central entity. CustomerID is the business key. CustomerType (Residential / Commercial / Industrial) drives tariff eligibility and DW segmentation. Status (Active / Inactive / Suspended) changes over time and is tracked via SCD-2 in DimCustomer. Has ModifiedAt for incremental ETL.

Tariff

Rate plan definition. Each row defines peak and off-peak rates per kWh plus a MonthlyFixedCharge. PeakHoursStart / PeakHoursEnd tell the ETL when to mark a reading as IsPeak. ExpiryDate = NULL means the tariff is currently active. Insert-only — extracted by CreatedAt.

ServiceAccount

The contractual link between a Customer and a service address. One customer can hold many service accounts. TariffID is held here so tariff changes are tracked by the SCD-2 process in DimServiceAccount. Has ModifiedAt.

Meter

The physical device. SerialNumber is the natural key. Status (Active / Faulty / Decommissioned) changes over time and is tracked via SCD-2 in DimMeter. Has ModifiedAt.

2.3 High-Volume Tables

MeterReading

The highest-volume table — 365,000 rows in development, millions in production. One row per daily reading per meter. KWhConsumed is stored explicitly (not computed) because SQL Server cannot persist a cross-row computed column. IsPeak is set at insert time. AnomalyFlag is set by the nightly ETL job. No ModifiedAt — readings are immutable. Extracted by CreatedAt.

CustomerMeter

Meter-to-account assignment journal. Tracks which meter was connected to which service account and when. IsCurrent = 1 means the link is active. Closing a link: set EndDate + IsCurrent = 0. Opening a new link: insert a new row with IsCurrent = 1. Has ModifiedAt with trigger.

2.4 Finance Tables

Billing

One row per ServiceAccount per billing month. The unique constraint on (ServiceAccountID, BillingPeriodStart) prevents duplicate bills. Status lifecycle: Unpaid → Paid / Overdue / Disputed / Partial. TotalAmount includes 8% tax. Has ModifiedAt so SSIS detects status changes.

Payment

Multiple payments can settle one bill (partial pays). PaymentMethod: CreditCard, BankTransfer, Cash, Online, Cheque. Immutable after creation — extracted by CreatedAt. Refunds create a new row with Status = Refunded.

BillingDispute

Dispute lifecycle: Open → UnderReview → Resolved / Rejected. ResolutionDate and ResolutionNotes are filled when closed. Has ModifiedAt so SSIS captures resolution updates.

2.5 Incremental Load Strategy per Table

Table	Watermark Column	Reason
Customer	ModifiedAt	Status and address changes must be detected
Tariff	CreatedAt	Insert-only — new tariffs only ever added
ServiceAccount	ModifiedAt	Tariff and status changes must be detected
Meter	ModifiedAt	Status and maintenance date changes
CustomerMeter	ModifiedAt	IsCurrent flips — trigger updates ModifiedAt
MeterInstallation	ModifiedAt	Corrections tracked via trigger
MeterMaintenance	ModifiedAt	Note corrections tracked via trigger
MeterReading	CreatedAt	Insert-only — readings never edited
Billing	ModifiedAt	Status changes Unpaid→Paid must be detected
Payment	CreatedAt	Insert-only — payments never edited
BillingDispute	ModifiedAt	Resolution updates must be detected
Notification	CreatedAt	Insert-only — messages never edited

Section 3

Phase 2 — ODS / Staging Database

3. ODS / Staging — EnergyUtility_StagingDB

The staging database is the raw landing zone. SSIS Package 1 extracts incremental rows from the OLTP source and writes them here before any transformation occurs. The staging layer acts as a safe, auditable copy of the delta that can be re-processed without touching the OLTP source again.

3.1 Design Principles

- No primary keys or foreign keys on Raw tables — data lands fast, validation happens in the DW load step.
- Every Raw table mirrors its OLTP source exactly, plus four audit columns.
- ETL_Watermark drives all incremental extracts — SSIS reads LastExtractedValue, extracts WHERE ModifiedAt > that value, then updates it on success.
- ETL_RunLog gives full auditability — every package run writes a row with row counts, status, and error messages.

3.2 Audit Columns on Every Raw Table

Column	Type	Purpose
ETL_RunID	INT	Links the row to the ETL_RunLog run that loaded it
ETL_LoadDate	DATETIME	When this row arrived in staging
ETL_Source	VARCHAR(100)	Source system name (EnergyUtility_TransactionDB)
RowStatus	VARCHAR(20)	New = first time seen, Changed = source row was modified

3.3 ETL Control Tables

ETL_Watermark

One row per OLTP table. SSIS reads LastExtractedValue at package start, extracts rows with ModifiedAt (or CreatedAt) greater than that value, then updates LastExtractedValue to GETDATE() on success. If a package fails, the watermark is not updated, so the next run re-extracts everything since the last success — no gaps, no duplicates.

ETL_RunLog

One row per package execution. Columns: RunID (identity), PackageName, StartTime, EndTime, RowsExtracted, RowsInserted, RowsUpdated, RowsRejected, Status (Running / Success / Failed), ErrorMessage. The stored proc usp_ETL_StartRun inserts the row and returns the RunID via OUTPUT parameter. usp_ETL_EndRun closes it with the final status.

Section 4

Phase 3 — Data Warehouse (Star Schema)

4. Data Warehouse — EnergyUtility_DataWarehouse

The data warehouse uses a star schema — flat dimension tables surrounding fact tables. Star schema is chosen over snowflake because analysts join Customer and ServiceAccount together on nearly every query; normalising them into a snowflake would add one join to every OLAP query with no meaningful benefit at this scale.

4.1 SCD Strategy

Dimension	SCD Type	Rationale
DimCustomer	Type 2	Address, city, status, and type changes must be preserved so historical facts remain accurate
DimServiceAccount	Type 2	Tariff changes on an account must create a new version
DimMeter	Type 2	Status changes (Active→Faulty→Decommissioned) must be versioned
DimTariff	Type 1	Rates are reference data; history not needed for bill restatement in this design
DimDate	None	Pre-populated static calendar 2019–2030 — never changes
DimBillingCycle	None	Pre-populated 2020–2030 billing months — never changes

4.2 SCD Type 2 Columns (on Customer, ServiceAccount, Meter)

- EffectiveDate DATE — the date this version became active
- ExpiryDate DATE NULL — NULL means this is the current version
- IsCurrent BIT — 1 for the active row, 0 for historical rows
- DW_InsertDate DATETIME — when SSIS first wrote this row
- DW_UpdateDate DATETIME NULL — when SSIS last expired this row

4.3 DimDate — Pre-populated Calendar

DimDate is populated by the 04_DW_Schema.sql script itself — no SSIS needed. It covers 2019-01-01 through 2030-12-31 (4,383 rows). DateKey uses the YYYYMMDD integer format (e.g. 20240115) so fact-to-dimension joins are integer comparisons — the fastest possible join type in SQL Server.

Key columns: DateKey, FullDate, DayOfWeek, DayName, DayOfMonth, DayOfYear, WeekOfYear, IsWeekend, MonthNumber, MonthName, Quarter, QuarterName, Year, FiscalYear, IsHoliday. IsHoliday defaults to 0 — update it with your region's public holidays before deploying to production.

4.4 FactEnergyUsage

Column	Type	Description
EnergyUsageKey	BIGINT IDENTITY	Surrogate PK
DateKey	INT FK	Links to DimDate (YYYYMMDD)
MeterKey	INT FK	Links to DimMeter (current version)
ServiceAccountKey	INT FK	Links to DimServiceAccount
CustomerKey	INT FK	Shortcut dimension — avoids join via ServiceAccount
TariffKey	INT FK	Links to DimTariff
ReadingType	VARCHAR(20)	Degenerate dim: Actual / Estimated / CustomerSubmitted
IsPeak	BIT	Degenerate dim: 1 if reading fell in peak hours
AnomalyFlag	BIT	Degenerate dim: 1 if reading flagged as anomalous
SourceMeterReadingID	BIGINT	OLTP PK for lineage and incremental guard
KWhConsumed	DECIMAL(12,4)	Measure: energy consumed this reading
EnergyCost	DECIMAL(12,2)	Measure: KWhConsumed x applicable rate

Column	Type	Description
ReadingHour	TINYINT	0-23 hour extracted from ReadingDate (no time dim needed)

Grain: one row per MeterReading record from the OLTP source. ~365,000 rows in development; millions in production. The CustomerKey shortcut dimension lets analysts slice by customer without joining through ServiceAccount on every query.

4.5 FactBilling

Grain: one row per ServiceAccount per BillingCycle (monthly). The MERGE strategy means when a bill status changes (Unpaid → Paid) the fact row is updated in place rather than duplicated. Key persisted computed columns:

- $\text{EnergyCharge} = \text{TotalAmount} - \text{TaxAmount} - \text{FixedCharges}$
- $\text{OutstandingBalance} = \text{TotalAmount} - \text{TotalPaymentsAmount}$
- IsOverdue, IsLate, HasDispute, DisputeCount — pre-computed flags for fast SSAS measure aggregation
- NumberOfReadings — count of MeterReadings in the billing period, joined during ETL
- DaysToPayment — DATEDIFF between BillingDate and first payment (NULL if not yet paid)

Section 5

Phase 4 — Billing Data Mart

5. Billing Data Mart — EnergyUtility_DataMart_Billing

The data mart is a finance-optimised subset of the DW. The finance team works exclusively with this mart — they never touch FactEnergyUsage or the raw reading data. The mart is simpler, faster for finance queries, and can be tuned independently.

5.1 Key Differences from the DW

Feature	Data Warehouse	Data Mart
DimCustomer	SCD-2 with history rows	Flat current-state only (IsCurrent=1)
FactBilling	Normalized — joins needed	CustomerType + City denormalized inline
Audience	SSAS + data analysts	Finance team Excel reports
Join complexity	Dim lookups required	Most queries are single-table scans
Population	SSIS Packages 2 + 3	SSIS Package 4 (reads from DW, not OLTP)

5.2 Tables

- DM_DimCustomer — one row per active customer, CustomerKey matches DW surrogate for traceability
- DM_DimBillingCycle — identical to DW DimBillingCycle, copied so the mart is self-contained
- DM_FactBilling — CustomerType, City, and State denormalized inline so slice-and-dice queries need zero joins

The most common finance query ("show me overdue invoices by customer type this quarter") is a single-table scan on DM_FactBilling with no joins. This is the design goal of the mart layer.

Section 6

Phase 5 — SSIS ETL Pipeline

6. SSIS ETL Pipeline — Four Packages

All four packages use Execute SQL Tasks calling stored procedures — not Data Flow Tasks. This is intentional: set-based SQL in stored procedures is 10-20x faster than row-by-row SSIS data flow processing for this domain. The packages are thin orchestration wrappers; all logic lives in the SQL layer where it is easier to test, debug, and version-control.

6.1 Package 1 — OLTP to Staging

What it does

Extracts incremental rows from all 12 OLTP tables into the corresponding Raw tables in StagingDB. Uses the ETL_Watermark table to determine what has changed since the last successful run.

Control flow — task sequence

1. Execute SQL: Start ETL Run — calls usp_ETL_StartRun, captures RunID into package variable User::RunID via Result Set = Single Row.
2. Execute SQL: Stage Customer — calls usp_Stage_Customer @RunID = ?
3. Execute SQL: Stage Tariff — calls usp_Stage_Tariff @RunID = ?
4. Execute SQL: Stage ServiceAccount — calls usp_Stage_ServiceAccount @RunID = ?
5. Execute SQL: Stage Meter — calls usp_Stage_Meter @RunID = ?
6. Execute SQL: Stage CustomerMeter — calls usp_Stage_CustomerMeter @RunID = ?
7. Execute SQL: Stage MeterInstallation — calls usp_Stage_MeterInstallation @RunID = ?
8. Execute SQL: Stage MeterMaintenance — calls usp_Stage_MeterMaintenance @RunID = ?

9. Execute SQL: Stage MeterReading — calls usp_Stage_MeterReading @RunID = ? (30-90 sec first run — normal)
10. Execute SQL: Stage Billing — calls usp_Stage_Billing @RunID = ?
11. Execute SQL: Stage Payment — calls usp_Stage_Payment @RunID = ?
12. Execute SQL: Stage BillingDispute — calls usp_Stage_BillingDispute @RunID = ?
13. Execute SQL: Stage Notification — calls usp_Stage_Notification @RunID = ?
14. Execute SQL: End Run Success — calls usp_ETL_EndRun @Status = 'Success'

Parameter mapping for every staging task (Tasks 2-13)

Field	Value
Variable Name	User::RunID
Direction	Input
Data Type	LONG (OLE DB mapping for SQL INT)
Parameter Name	0 (position of the ? in the SQL)
Parameter Size	-1

Error handling

Add red failure arrows from every task to a final "End Run Failed" Execute SQL Task that calls usp_ETL_EndRun with @Status = 'Failed'. Set ForceExecutionResult = Failure on that task so SQL Server Agent marks the job as failed and sends alert emails.

6.2 Package 2 — Staging to Dimensions (SCD-2)

Calls usp_Load_AllDimensions which runs four procs in dependency order: DimTariff (SCD-1) → DimCustomer (SCD-2) → DimServiceAccount (SCD-2) → DimMeter (SCD-2). Order matters because DimServiceAccount needs CustomerKey from DimCustomer and TariffCode from DimTariff.

SCD-2 MERGE logic (two-step)

- Step 1 (UPDATE): expire rows whose tracked attributes have changed. Set ExpiryDate = today, IsCurrent = 0 where source row differs from current DW row.
- Step 2 (INSERT): insert a new version for changed rows AND brand-new rows. Insert only where no IsCurrent = 1 row already exists for that business key.

6.3 Package 3 — Staging to Facts

FactEnergyUsage

Joins MeterReadingRaw to DimMeter, DimServiceAccount, DimCustomer, DimTariff, DimDate. Computes EnergyCost = KWhConsumed x applicable rate. Uses an incremental guard: WHERE NOT EXISTS (SELECT 1 FROM FactEnergyUsage WHERE SourceMeterReadingID = mr.MeterReadingID) to prevent duplicates on re-runs.

FactBilling

Uses SQL MERGE on SourceBillingID. When a bill already exists and its status has changed (Unpaid → Paid), MERGE updates the existing row. Pre-aggregates payments and disputes into temp tables (#PaymentAgg, #DisputeAgg, #ReadingCount) before the MERGE to keep the main statement clean and set-based.

6.4 Package 4 — DW to Data Mart

Calls usp_Load_DataMart which runs three procs: usp_DM_Load_BillingCycle → usp_DM_Load_Customer → usp_DM_Load_FactBilling. All three use MERGE so re-runs are safe. CustomerType, City, and State are denormalized into DM_FactBilling during this step.

6.5 Verification Queries After Each Package

After Package 1

```
USE EnergyUtility_StagingDB;
SELECT TableName, LastExtractedValue FROM ETL_Watermark;
SELECT COUNT(*) FROM CustomerRaw;          -- expect ~1000
SELECT COUNT(*) FROM MeterReadingRaw;      -- expect ~365000
SELECT * FROM ETL_RunLog ORDER BY RunID DESC;
```

After Package 2

```
SELECT COUNT(*) FROM DimCustomer WHERE IsCurrent=1;      -- 1000
SELECT COUNT(*) FROM DimServiceAccount WHERE IsCurrent=1; -- 1000
SELECT COUNT(*) FROM DimMeter WHERE IsCurrent=1;         -- ~995
SELECT COUNT(*) FROM DimTariff WHERE IsActive=1;         -- 5
```

After Package 3

```
SELECT COUNT(*) FROM FactEnergyUsage;           -- ~365000
SELECT COUNT(*) FROM FactBilling;                -- ~12000
SELECT SUM(TotalAmount) FROM FactBilling;        -- non-zero
```

After Package 4

```
SELECT COUNT(*) FROM DM_FactBilling;             -- ~12000
SELECT SUM(TotalAmount) FROM DM_FactBilling;     -- matches FactBilling
total
```

Section 7

Phase 6 — SSAS Multidimensional Cube

7. SSAS Multidimensional Cube — EnergyUtility_SSAS

The SSAS project contains two cubes sharing six dimensions. Both cubes connect to EnergyUtility_DataWarehouse as the data source. SSAS pre-aggregates the data into MOLAP storage on disk so Excel and Power BI queries return in milliseconds instead of the seconds a SQL query would take against the raw fact tables.

7.1 Build Order in Visual Studio

15. New project: Analysis Services Multidimensional and Data Mining Project
16. Data Source: EnergyUtility_DW pointing to EnergyUtility_DataWarehouse
17. Data Source View: include all 8 tables + add Named Query DimDueDate (copy of DimDate for DueDateKey role in FactBilling)
18. Build six dimensions (see 7.2)
19. Build EnergyUsageCube based on FactEnergyUsage
20. Build BillingCube based on FactBilling
21. Configure Dimension Usage tab for both cubes (see 7.3)
22. Deploy project to Analysis Services
23. Process Full — populates the cube from DW
24. Browse cube in Visual Studio to verify measures return values

7.2 Six Dimensions

Dimension	Key Column	Name Column	SCD	Key Hierarchies
DimDate	DateKey	FullDate	None	Calendar: Year > Quarter > Month > Day
DimBillingCycle	BillingCycleKey	BillingMonthName	None	BillingCalendar: Year > Quarter > Month

Dimension	Key Column	Name Column	SCD	Key Hierarchies
DimCustomer	CustomerKey	FullName	Type 2	Geography: State > City > Zip > Name
DimServiceAccount	ServiceAccountKey	AccountNumber	Type 2	Location: State > City > AccountNumber
DimTariff	TariffKey	TariffName	Type 1	None (5 rows only — use attribute filter)
DimMeter	MeterKey	SerialNumber	Type 2	Category: Type > Manufacturer > Serial

7.3 Dimension Usage — EnergyUsageCube

Dimension	Relationship	Fact FK Column
DimDate	Regular	DateKey
DimMeter	Regular	MeterKey
DimServiceAccount	Regular	ServiceAccountKey
DimCustomer	Regular	CustomerKey
DimTariff	Regular	TariffKey
DimBillingCycle	No relationship	(not in FactEnergyUsage)

7.4 Dimension Usage — BillingCube

Dimension	Relationship	Fact FK Column
DimBillingCycle	Regular	BillingCycleKey
DimCustomer	Regular	CustomerKey
DimServiceAccount	Regular	ServiceAccountKey
DimTariff	Regular	TariffKey

Dimension	Relationship	Fact FK Column
DimDate	Regular	BillingDateKey
DimMeter	No relationship	(billing is at account level)

7.5 MDX Calculated Members — EnergyUsageCube

Measure Name	Formula Summary	Format
Avg Daily KWh Per Customer	KWh Consumed / Reading Count	#,##0.00
Peak KWh	KWh Consumed filtered IsPeak = 1	#,##0.00
Off-Peak KWh	KWh Consumed - Peak KWh	#,##0.00
Peak Usage %	Peak KWh / KWh Consumed	0.00%
Blended Rate Per KWh	Energy Cost / KWh Consumed	\$#,##0.0000
KWh MoM Change	Current - ParallelPeriod(1 month)	#,##0.00
KWh MoM Change %	KWh MoM Change / prior month	0.0%
KWh YTD	SUM over YTD members	#,##0.00
KWh 3M Rolling Avg	SUM LastPeriods(3) / 3	#,##0.00

7.6 MDX Calculated Members — BillingCube

Measure Name	Formula Summary	Format
Collection Rate %	Total Payments Amount / Total Amount	0.0%
Avg Outstanding Per Bill	Outstanding Balance / Bill Count	\$#,##0.00
Overdue Rate %	Is Overdue (SUM) / Bill Count	0.0%

Measure Name	Formula Summary	Format
Dispute Rate %	Has Dispute (SUM) / Bill Count	0.0%
Late Payment Rate %	Is Late (SUM) / Bill Count	0.0%
Avg Days To Payment	Days To Payment / Payment Count	0.0
Revenue Per Customer	Total Amount / Customer count	\$\$,##0.00
Revenue MoM Change	Current - ParallelPeriod(1 month)	\$\$,##0.00
Revenue MoM Change %	Revenue MoM Change / prior month	0.0%
Revenue YTD	SUM over YTD billing cycle members	\$\$,##0.00

7.7 KPIs (BillingCube)

KPI Name	Value Measure	Target	Green	Amber	Red
Collection Rate	Collection Rate %	90%	>= 90%	75-90%	< 75%
Overdue Rate	Overdue Rate %	10%	<= 10%	10-12%	> 12%
Dispute Rate	Dispute Rate %	5%	<= 5%	5-8%	> 8%

7.8 Nightly Cube Refresh

After each SSIS run, the cube must be reprocessed. Add a SQL Server Agent job step of type "SQL Server Analysis Services Command" with this XMLA:

```
<Batch xmlns="http://schemas.microsoft.com/analysiservices/2003/engine">
  <Process>
    <Object><DatabaseID>EnergyUtility_SSAS</DatabaseID></Object>
    <Type>ProcessFull</Type>
  </Process>
</Batch>
```

Use ProcessFull during development. Switch to ProcessUpdate in production — it is incremental and much faster once the cube is built.

Section 8

Phase 7 — Excel and Power BI Dashboards

8. Excel Dashboard

Connect via Data → Get Data → From Analysis Services → enter server name → select EnergyUtility_SSAS. Create two connections — one per cube. Every PivotTable must use "Connect live" mode; never import data directly into Excel.

8.1 Workbook Structure

Sheet	Connection	Key Visuals
Executive Summary	BillingCube + EnergyUsageCube	6 KPI metric tiles, Monthly revenue trend, kWh by customer type
Energy Consumption	EnergyUsageCube	Daily trend, Peak/Off-peak donut, Top 10 accounts bar, Anomaly heatmap
Billing & Collections	BillingCube	Revenue waterfall, Status pie, Outstanding by segment, Monthly KPI table
Disputes & Risk	BillingCube	Dispute rate bars, Risk trend, High-risk customer list, City risk summary

8.2 Slicer Connection Rule

Every slicer on a sheet must be connected to ALL PivotTables on that sheet. Right-click each slicer → Report Connections → check all tables. Without this, slicers only filter the table they were created from and analysts get inconsistent cross-visuals.

8.3 Conditional Formatting for KPI Columns

Column	Green	Amber	Red
Collection Rate %	> 90%	75-90%	< 75%

Column	Green	Amber	Red
Overdue Rate %	< 8%	8-12%	> 12%
Dispute Rate %	< 3%	3-5%	> 5%

8.4 Auto-Refresh Setup

Data tab → Connections → select each connection → Properties. Check "Refresh data when opening the file" and "Enable background refresh". When the finance team opens the workbook each morning, it automatically pulls the latest data from the SSAS cube — which was already refreshed by the 2 AM SQL Agent job.

9. Power BI Dashboard

Power BI connects via Live Connection (never Import mode). Create two .pbix files — one per cube. Install the On-premises data gateway on the Analysis Services machine to enable cloud sharing via Power BI Service.

9.1 Report Pages per File

EnergyUsage_Report.pbix

- Page 1: Consumption Overview — cards, monthly line, segment bar, detailed matrix
- Page 2: Peak Load Analysis — stacked area, anomaly scatter, geographic map, top-10 bar

Billing_Report.pbix

- Page 1: Revenue Dashboard — KPI visuals with traffic lights, revenue column chart, status donut, monthly table
- Page 2: Collections & Risk — funnel chart, outstanding by city bar, risk matrix scatter, high-risk customer table
- Page 3: Tariff Analysis — revenue by tariff bar, tariff x segment matrix, tariff trend line

9.2 Automated Full Refresh Chain

Time	Step	Tool
2:00 AM	Run Package1 — extract 12 OLTP tables to staging	SQL Agent → SSIS
2:05 AM	Run Package2 — SCD-2 dimension loads	SQL Agent → SSIS
2:08 AM	Run Package3 — FactEnergyUsage + FactBilling	SQL Agent → SSIS
2:12 AM	Run Package4 — billing data mart refresh	SQL Agent → SSIS
2:14 AM	ProcessFull SSAS cube — rebuilds from fresh DW data	SQL Agent → XMLA
9:00 AM	Finance opens Excel — auto-refresh fires on open	Excel connection
Any time	Analyst opens Power BI — Live Connection queries cube	Power BI Service

Section 9

Complete Run Order and Troubleshooting Reference

10. Complete Run Order

10.1 One-time Setup (SSMS — run scripts in this exact order)

25. Connect to EnergyUtility_TransactionDB → run 01_OLTP_Schema.sql
26. Still in TransactionDB → run 02_OLTP_SeedData_v2.sql (takes 2-5 min for MeterReading)
27. Connect to EnergyUtility_StagingDB → run 03_Staging_Schema.sql
28. Connect to EnergyUtility_DataWarehouse → run 04_DW_Schema.sql (populates DimDate automatically)
29. Connect to EnergyUtility_DataMart_Billing → run 05_DataMart_Schema.sql
30. Connect to EnergyUtility_StagingDB → run 06_Package1_Staging_SPs.sql
31. Connect to EnergyUtility_DataWarehouse → run 07_Package2_Dims_SPs.sql
32. Connect to EnergyUtility_DataWarehouse → run 08_Package3_Facts_SPs.sql
33. Connect to EnergyUtility_DataMart_Billing → run 09_Package4_DataMart_SPs.sql

10.2 SSIS (Visual Studio — run packages in this order after building them)

34. Execute Package1_OLTP_to_Staging.dtsx → verify CustomerRaw ~1000, MeterReadingRaw ~365000
35. Execute Package2_Dims.dtsx → verify DimCustomer IsCurrent=1 count = 1000
36. Execute Package3_Facts.dtsx → verify FactEnergyUsage ~365000, FactBilling ~12000
37. Execute Package4_DataMart.dtsx → verify DM_FactBilling ~12000

10.3 SSAS (Visual Studio)

38. Build the SSAS project (Build → Build EnergyUtility_SSAS)
39. Deploy (Build → Deploy EnergyUtility_SSAS)
40. Process Full (right-click project → Process → Process Full)
41. Browse each cube in the Browser tab to verify measures return values

10.4 SQL Server Agent Job

42. Create job: EnergyUtility_DailyETL
43. Step 1: Run Master_ETL.dtsx (Execute Package Task chaining all four packages)
44. Step 2: ProcessFull SSAS (Analysis Services Command, XMLA above)
45. Schedule: Daily at 2:00 AM
46. Notifications: email operator on failure

10.5 Troubleshooting Common Errors

Error	Cause	Fix
SSIS Task 1 fails: RunID = 0 downstream	ResultSet set to None instead of Single Row	Open Task 1 editor → General tab → change ResultSet to "Single row"
Staging proc not found	06_Package1_Staging_SPs.sql not run in StagingDB	Run the script, confirm with: SELECT name FROM sys.procedures
DimServiceAccount insert fails	DimCustomer loaded before DimServiceAccount	Check usp_Load_AllDimensions runs Tariff → Customer → ServiceAccount → Meter
FactEnergyUsage zero rows	Dimension Usage FK columns misconfigured in SSAS	In cube designer → Dimension Usage tab → verify each FK column
Cross-database permission error in SSIS	Service account lacks SELECT on TransactionDB	Grant SELECT to the SSIS service account on EnergyUtility_TransactionDB
SSAS Deploy fails: server not found	Analysis Services instance name wrong	Right-click SSAS project → Properties → Deployment tab → correct the Server name
Excel PivotTable shows no data	Wrong ResultSet or cube not processed	Verify cube processed successfully; re-connect via Data → Get Data → Analysis Services

Appendix

Skills Practised and Technology Reference

Appendix A — Skills Practised

SQL Server Management Studio (SSMS)

- Designing a complex 12-table OLTP schema with FK constraints, check constraints, and default values
- Implementing SCD-2 patterns using MERGE statements
- Writing set-based seed data using CTEs and window functions (SUM OVER ROWS BETWEEN)
- Cross-database queries (three-part name: Database.Schema.Table)
- AFTER UPDATE triggers for ModifiedAt maintenance

SSIS (Visual Studio SSDT)

- Execute SQL Task configuration with OLE DB connections and parameter mapping
- Package-level variables and Result Set capture
- Watermark-based incremental loading pattern
- Green success and red failure precedence constraints
- ForceExecutionResult for SQL Server Agent integration
- Master package chaining via Execute Package Task

Data Modelling

- Star schema design: fact grain selection, surrogate keys, degenerate dimensions
- SCD Type 1 vs Type 2 trade-off decisions
- Shortcut dimensions to reduce join depth
- Persisted computed columns for pre-calculated measures
- ETL control framework: watermark tables, run logs, idempotent reprocessing

SSAS Multidimensional

- Data Source and Data Source View setup
- Named Queries for role-playing dimensions (DimDueDate)

- Dimension attribute relationships and user-defined hierarchies
- Marking dimensions as Date Tables for time intelligence
- Cube design: measure groups, dimension usage, regular relationships
- MDX calculated members: DIVIDE, PARALLELPERIOD, YTD, LASTPERIODS, FILTER, TOPCOUNT
- KPI configuration with status and trend expressions
- Named sets for analyst-friendly filtering
- XMLA ProcessFull and ProcessUpdate commands

Excel and Power BI

- Analysis Services live connection (not import)
- PivotTable and PivotChart design with multiple measures and hierarchies
- Slicer cross-report connections
- Conditional formatting for KPI traffic lights
- Power BI gateway for on-premises SSAS connectivity
- Power BI KPI visual with target and trend axis

Appendix B — Seed Data Distributions

Table	Rows	Distribution
Tariff	5	2 Residential, 2 Commercial, 1 Industrial
Customer	1,000	60% Residential, 30% Commercial, 10% Industrial across 10 US cities
Meter	1,000	33% Smart, 33% Digital, 33% Analog; 1% Faulty, 0.5% Decommissioned
ServiceAccount	1,000	1:1 with customers; tariff assigned by customer type
CustomerMeter	1,000	1:1 with service accounts; all IsCurrent = 1
MeterInstallation	1,000	1:1 with CustomerMeter; 5 rotating technicians
MeterMaintenance	200	4 types: Calibration, Inspection, Repair, Replacement
MeterReading	~365,000	365 days x 995 active meters; 40% peak, 2% anomaly, 5% estimated

Table	Rows	Distribution
Billing	~12,000	12 months x 1000 accounts; 70% Paid, 12% Unpaid, 10% Overdue, 5% Partial, 3% Disputed
Payment	~8,500	One per Paid/Partial bill; 5 payment methods; paid 0-19 days after billing date
BillingDispute	~360	3% of billing records; 4 statuses; 5 dispute reason categories
Notification	~3,000	3 channels x 5 types across all customers